Mini project: Intelligent vacuum cleaning agent.

Course: Csc 3309 Introduction to Artificial Intelligence.

Date:11/02/2026.

Team members:

Doha Ilyass

Omar El Mokhtar El Bousouni

Salma Madoud

Zineb Akhouad

1_Description:

This mini project focuses on designing an intelligent vacuum cleaning agent that operates in a linear environment of size n_rooms. The environment consists of a number of rooms, where each room has a varying dirtiness level, from 0 to 5,0 meaning that the room is clean, 1 means the least dirty, and 5 means the most dirty.

2_Environment Design:

2.1_Room representation:

```python
class Room:
    def __init__(self, dirtiness_level: int = 0):
        self.dirtiness_level = dirtiness_level

    def is_clean(self) -> bool:
        return not self.dirtiness_level

    def clean(self) -> None:
        self.dirtiness_level = 0

    def make_dirty(self, level: int) -> None:
        if 1 <= level <= 5:
            self.dirtiness_level = level
        else:
            raise ValueError("Dirt level must be between 1 and 5")

    def get_dirt_level(self) -> int:
        return self.dirtiness_level
```

- Each room has a dirtiness_level attribute initialized with a random number between 0 and 5(cleanest to dirtiest).
- Room consists of the following methods:

1. is_clean( ): which checks whether the room is clean.
2. clean(): which sets dirtiness level to zero.
3. make_dirty( ): which sets the dirtiness level to a random number from 1 to 5.
4. Get_dirty_level( ): which returns the dirtiness level.

2.2_Environment Representation:

The environment Class contains:

- n_rooms : number of rooms.
- rooms: list of Room objects.
- random.randint(0, 5) is used to initialize the environment, where each room is assigned a random dirtiness level between 0 and 5.

The environment consists of the following functions:

- Access a specific room.
- Clean a room.
- Check if all rooms are clean.
- Return the dirtiness level of all rooms.

2.3_Dirtiness update:

The project dynamically updates the environment following an update rule, which is :

- At every time step, every clean room has a probability of 10% of becoming dirty again.
- The new dirtiness level is assigned randomly between 1 and 5.

This is implemented using the following method:

```
def update_environment(self) -> None:
    for room in self.rooms:
        if room.is_clean() and random.random() < 0.1:
            room.make_dirty(random.randint(1, 5))
```

3_Agent Design Decisions:

3.1_Agent initialization:

The agent class contains the following attributes:

- current_position: which starts at room 0.
- current_energy: which is initialized to 2.5*n_rooms (n_rooms being the number of rooms).
- logs: stores all the actions performed by the agent.
- direction: used for movement, either left or right.

3.2_Perception Model:

The agent uses the perceive( ) method to analyze the environment. The percepts are :

- current room: the current position.
- state of the room: whether it is clean or dirty.
- Dirtiness level of the current room.
- remaining energy.

3.3_Action Set:

The agent has three possible actions to perform, which are the following :

Suck: cleans the current room.

MoveLeft: moves to the previous room.

MoveRight: Move to the next room.

The energy costs depend on the actions taken:

+suck=dirtiness level(1 to 5)

+Move cost=2 energy units.

3.4_ Possibility of performing an action:

To ensure that the agent follows the energy usage rules properly, the agent uses the following methods:

```
def can_suck(self, environment: Environment) -> bool:
    current_room = environment.get_room(self.current_position)
    return current_room.get_dirt_level() <= self.current_energy

def can_move(self) -> bool:
    return 2 <= self.current_energy
```

- Can_suck( ): checks if the dirtiness cost <=the remaining energy.
- Can_move( ): checks if the remaining energy is>=2.

4_Agent Behavior Summary:

4.1_Agent decision rules:

The agent follows a simple strategy. It observes the current room and its remaining energy, then performs an action based on the rules.

Rule 1:Clean the room if it is dirty and energy is sufficient:

```
if not current_room.is_clean() and self.can_suck(environment):
    return 'Suck'
```

Rule 2:Stop when all the rooms are clean:

```
# Rule 2: Mission complete - all rooms clean
if environment.is_all_clean():
    return 'Stop'
```

Rule 3:If energy is sufficient, move either left ot right depending on the position:

```python
# Rule 3: Try to move with direction tracking
if self.can_move():
    if self.current_position == self.n_rooms - 1:
        self.direction = 'left'
        return 'MoveLeft'
    elif self.current_position == 0:
        self.direction = 'right'
        return 'MoveRight'
    else:
        # Continue in current direction
        if self.direction == 'right':
            return 'MoveRight'
        else:
            return 'MoveLeft'
```

Rule 4:The agent stops when it cannot further afford to either clean or move.

```python
return 'Stop'
```

4.2_Simulation:

The simulation runs for a maximum number of steps. At each step, the agent perceives the environment, decides an action, executes it, and then the environment may update by randomly making clean rooms dirty. The simulation stops when all rooms are clean, when the agent runs out of usable energy, or when the maximum number of steps is reached.

5_Performance observations:

The agent cleans dirty rooms while also respecting energy constraints. Its behavior is predictable. The bouncing strategy makes sure that the agent properly explores the environment because some rooms might become dirty again.

During the simulation, the agent records all the actions performed along with the energy costs. Performance is assessed by the remaining energy, the number of clean rooms, and the final state of the environment. However, the agent might waste energy when moving to check already clean rooms because it does not use advanced planning to minimize the travel distance.

6_Conclusion:

The project successfully develops an intelligent cleaning agent in a linear environment. It simulates a realistic environment by handling random dirt generation and by operating with limited energy, depending on the number of rooms.

Future improvements could include prioritizing dirty rooms and planning to determine the shortest distance to move from one room to the other, to improve performance and energy efficiency.